

Solving ODEs with Octave 11.3.0

A GUI-based guide for the ODE module

This guide accompanies the eight Octave scripts in the Octave Code/ folder. It assumes Octave 11.3.0 (GUI) is already installed on your system and that you have access to the command window, editor, and plot window. By the end of this guide, you will be able to solve every type of first-order ODE taught in the slide deck — separable, linear, exact, non-exact, and numerical — using Octave, and verify your analytical solutions by back-substitution.

CONTENTS

Chapter	Topic	Companion Script
1	GUI Basics — Command Window, Editor, Workspace	00_gui_basics.m
2	Symbolic ODE Solving — Separation	01_separable_ode.m
3	Symbolic ODE Solving — Linear ODEs	02_linear_ode.m
4	Symbolic ODE Solving — Exact Equations	03_exact_ode.m
5	Symbolic ODE Solving — Non-Exact with IF	04_nonexact_if.m
6	Numerical Methods — Euler, Heun, RK4	05_numerical_methods.m
7	Plotting — Slope Fields & Solution Curves	06_slope_fields.m
8	Verification — Back-Substitution	07_verification.m

GUI Basics

The Octave 11.3.0 GUI provides four primary work areas. Understanding what each one does is the foundation for everything that follows in this guide.

THE FOUR WORK AREAS

Area	Location	Purpose
Command Window	Bottom-left (default)	Interactive REPL. Type commands, see results immediately. Use this to experiment and
Workspace	Top-left	Lists all current variables with their values, sizes, and classes. Inspect variables you ha
File Browser	Left sidebar	Navigate folders, open .m files in the Editor, set the working directory.
Editor	Center	Write and edit .m scripts. Provides syntax highlighting, indentation, and bracket matc
Plot Window	Separate window	Displays figures created by plot, fplot, quiver, contour, etc. Each figure is numbered (f

FIRST COMMANDS

Try these in the Command Window to get a feel for Octave syntax. Press Enter after each line.

```
% Basic arithmetic
3 + 4           % ans = 7
2^10           % ans = 1024
pi             % ans = 3.1416
exp(1)         % ans = 2.7183 (Euler's number)

% Variables
x = 5           % assign
y = [1, 2, 3]   % row vector
z = 1:0.5:3     % [1, 1.5, 2, 2.5, 3]

% A simple plot
x = linspace(0, 2*pi, 100);
y = sin(x);
plot(x, y, "b-", "LineWidth", 2);
title("y = sin(x)");
xlabel("x"); ylabel("y");
grid on;
```

LOADING THE SYMBOLIC PACKAGE

Octave does not include symbolic math by default. You need the symbolic package for analytical ODE solving (the dsolve function). Install it once, then load it each session.

```
% Install once (run from Command Window):  
pkg install -forge symbolic  
  
% Load each session (put at top of any .m script that uses dsolve):  
pkg load symbolic  
  
% Verify it works:  
syms x y  
f = x^2 + y^2  
diff(f, x)      % ans = 2*x  
int(f, x)       % ans = x^3/3 + x*y^2
```

Note: The symbolic package requires Python and SymPy to be installed on your system. If installation fails, see the Octave Wiki for troubleshooting. On Windows, the simplest route is to install Python from python.org (with the "Add to PATH" option checked), then run `pip install sympy` from a command prompt.

Symbolic ODE Solving — Separation

The `dsolve` function solves ODEs analytically. For a separable equation like $dy/dx = 2xy$, `dsolve` returns the general solution (or, with an initial condition, the particular solution).

THE PATTERN

```
pkg load symbolic

syms y(x) x          % declare y as a function of x

% Define the ODE: dy/dx = 2*x*y
ode = diff(y, x) == 2*x*y;

% Define the initial condition
cond = y(0) == 1;

% Solve
sol = dsolve(ode, cond);
disp(sol);           % displays: y(x) = exp(x^2)

% Simplify
sol = simplify(sol);
```

KEY POINTS

- Declare y as a function of x using `syms y(x) x`. This tells Octave that y depends on x , so `diff(y, x)` returns dy/dx rather than treating y as a constant.
- Use `==` for equations, not `=`. The single `=` is for assignment; `==` creates a symbolic equation object that `dsolve` can parse.
- The initial condition is also an equation: $y(0) == 1$. Pass it as a second argument to `dsolve`. For higher-order ODEs, pass multiple conditions.
- Always simplify. The raw `dsolve` output may contain equivalent but uglier forms (e.g., $\exp(x^2 + \log(1))$ instead of $\exp(x^2)$). The `simplify` function cleans these up.

PLOTTING THE SOLUTION

Use `fplot` (function plot) to graph a symbolic solution. Unlike `plot`, which takes numeric vectors, `fplot` takes a symbolic expression and a domain.

```
figure(1); clf;
fplot(sol, [0, 2], "b-", "LineWidth", 2);
hold on;
fplot(@(x) exp(x.^2), [0, 2], "r--", "LineWidth", 1.5);
title("Separable ODE: dy/dx = 2xy, y(0) = 1");
xlabel("x"); ylabel("y");
legend("Symbolic solution", "e^{x^2} (verification)", "Location", "northwest");
grid on;
```

Symbolic ODE Solving — Linear ODEs

Linear first-order ODEs of the form $y' + P(x)y = Q(x)$ are solved the same way as separable ODEs — pass the ODE and initial condition to `dsolve`. Octave handles the integrating factor automatically behind the scenes.

EXAMPLE

Solve $dy/dx + (2/x)y = 4x^2$ with $y(1) = 2$.

```
pkg load symbolic
syms y(x) x

% Define the ODE in standard form: y' + (2/x)y = 4*x^2
ode = diff(y, x) + (2/x)*y == 4*x^2;
cond = y(1) == 2;

sol = dsolve(ode, cond);
disp(simplify(sol)); % (2*x^6 + 4) / (3*x^2)
```

COMPUTING THE INTEGRATING FACTOR MANUALLY

For pedagogical purposes, you can compute the integrating factor $\mu = e^{\int P(x) dx}$ symbolically and verify the product-rule collapse:

```
P = 2/x;
mu = exp(int(P, x)); % mu = x^2
disp(mu);

% Verify: d/dx[mu * y] should equal mu * Q(x)
Q = 4*x^2;
lhs_check = diff(mu * sol, x);
rhs_check = mu * Q;
disp(simplify(lhs_check - rhs_check)); % 0 ✓
```

Symbolic ODE Solving — Exact Equations

For exact equations written in differential form $M dx + N dy = 0$, the workflow is different from `dsolve`. You test for exactness, find the potential F by integrating, and the solution is $F(x,y) = C$.

THE WORKFLOW

```
pkg load symbolic
syms x y

% Define M and N
M = 2*x*y + 3;
N = x^2 - 1;

% Step 1: Test for exactness
M_y = diff(M, y);
N_x = diff(N, x);
if simplify(M_y - N_x) == 0
    disp("Exact!");
else
    disp("Not exact. Need an integrating factor.");
end

% Step 2: Find F by integrating M in x
F_partial = int(M, x);           % x^2*y + 3*x

% Step 3: g'(y) = N - dF_partial/dy
g_prime = simplify(N - diff(F_partial, y));
g = int(g_prime, y);             % -y

% Step 4: F = F_partial + g(y)
F = F_partial + g;               % x^2*y + 3*x - y
disp(F);

% Solution: F(x, y) = C
```

PLOTTING LEVEL CURVES

The solution of an exact equation is the family of level curves $F(x,y) = C$. Use `contour` to plot them:

```
figure(1); clf;
[X, Y] = meshgrid(linspace(-2, 2, 100), linspace(-2, 2, 100));
F_vals = double(subs(F, {x, y}, {X, Y}));
contour(X, Y, F_vals, 20);
title("Level curves of F(x,y) = C");
xlabel("x"); ylabel("y");
colorbar; grid on;
```

Symbolic ODE Solving — Non-Exact with Integrating Factor

When $M dx + N dy = 0$ is not exact, you test for an x-only or y-only integrating factor. Octave can compute $(M_y - N_x)/N$ symbolically; you check whether the result contains y.

THE TESTS

```
% Compute (M_y - N_x) / N
test_x = simplify((M_y - N_x) / N);
fprintf("(M_y - N_x)/N = %s\n", char(test_x));

% Check if this is a function of x only (no y)
vars = symvar(test_x);
if isempty(vars) || ~ismember(y, vars)
    fprintf("x-only! mu(x) = exp(integrate this in x)\n");
    mu = exp(int(test_x, x));
else
    % Try y-only test
    test_y = simplify((N_x - M_y) / M);
    fprintf("(N_x - M_y)/M = %s\n", char(test_y));
    mu = exp(int(test_y, y));
end

fprintf("Integrating factor: %s\n", char(mu));
```

MULTIPLY AND SOLVE AS EXACT

Once you have the integrating factor, multiply M and N by it, verify the new equation is exact, and proceed with the exact-equation workflow from Chapter 4.

```
M_new = simplify(M * mu);
N_new = simplify(N * mu);

% Verify exactness
M_new_y = diff(M_new, y);
N_new_x = diff(N_new, x);
assert(simplify(M_new_y - N_new_x) == 0);

% Now solve as exact (see Chapter 4)
F_partial = int(M_new, x);
g_prime = simplify(N_new - diff(F_partial, y));
g = int(g_prime, y);
F = simplify(F_partial + g);
fprintf("Solution: %s = C\n", char(F));
```

Numerical Methods — Euler, Heun, RK4

For ODEs with no closed-form solution, use numerical stepping. Octave provides the built-in `ode45` (adaptive RK4/5), but for understanding the methods, it is valuable to implement Euler, Heun, and RK4 from scratch.

EULER'S METHOD (FROM SCRATCH)

```
f = @(x, y) x + y;      % dy/dx = x + y
x0 = 0; y0 = 1;         % initial condition
h = 0.1;                % step size
x_end = 0.3;
n_steps = round((x_end - x0) / h);

x_euler = x0; y_euler = y0;
for i = 1:n_steps
    y_euler(i+1) = y_euler(i) + h * f(x_euler(i), y_euler(i));
    x_euler(i+1) = x_euler(i) + h;
end
```

HEUN'S METHOD (PREDICTOR-CORRECTOR)

```
x_heun = x0; y_heun = y0;
for i = 1:n_steps
    y_pred = y_heun(i) + h * f(x_heun(i), y_heun(i));
    y_heun(i+1) = y_heun(i) + (h/2) * (...
        f(x_heun(i), y_heun(i)) + ...
        f(x_heun(i) + h, y_pred));
    x_heun(i+1) = x_heun(i) + h;
end
```

RK4 (CLASSICAL RUNGE-KUTTA)

```
x_rk4 = x0; y_rk4 = y0;
for i = 1:n_steps
    k1 = f(x_rk4(i), y_rk4(i));
    k2 = f(x_rk4(i) + h/2, y_rk4(i) + h/2 * k1);
    k3 = f(x_rk4(i) + h/2, y_rk4(i) + h/2 * k2);
    k4 = f(x_rk4(i) + h, y_rk4(i) + h * k3);
    y_rk4(i+1) = y_rk4(i) + (h/6) * (k1 + 2*k2 + 2*k3 + k4);
    x_rk4(i+1) = x_rk4(i) + h;
end
```

USING OCTAVE'S BUILT-IN `ode45`

For production work, use `ode45`. It adaptively adjusts the step size to maintain a specified tolerance, so you do not need to choose `h` manually.


```

f = @(x, y) x + y;
[x_ode45, y_ode45] = ode45(f, [0, 0.3], 1);

% Plot all three methods vs the exact solution
figure(1); clf;
x_fine = linspace(0, 0.3, 100);
plot(x_fine, 2*exp(x_fine) - x_fine - 1, "k-", "LineWidth", 2);
hold on;
plot(x_euler, y_euler, "ro-", "LineWidth", 1.5);
plot(x_heun, y_heun, "gs-", "LineWidth", 1.5);
plot(x_rk4, y_rk4, "b^-", "LineWidth", 1.5);
legend("Exact", "Euler", "Heun", "RK4", "Location", "northwest");
grid on;

```

Plotting — Slope Fields & Solution Curves

A slope field (also called a direction field) shows the slope of the solution at every point in the plane. Use `quiver` to draw small arrows at each grid point, then overlay solution curves from `ode45`.

THE COMPLETE PATTERN

```
clear; clc; close all;

% Define the ODE
f = @(x, y) sin(x .* y);

% Create a grid of points
x_range = linspace(0, 3, 20);
y_range = linspace(-2, 4, 20);
[X, Y] = meshgrid(x_range, y_range);

% Compute slopes
DY = f(X, Y);
DX = ones(size(DY));

% Normalize arrow lengths for uniform appearance
L = sqrt(DX.^2 + DY.^2);
DX_norm = DX ./ L;
DY_norm = DY ./ L;

% Plot the slope field
figure(1); clf;
quiver(X, Y, DX_norm, DY_norm, 0.5, "Color", [0.5 0.5 0.5], "LineWidth", 1);
hold on;

% Overlay solution curves from several initial conditions
initial_ys = [-1, 0, 1, 2, 3];
for i = 1:length(initial_ys)
    [x_sol, y_sol] = ode45(f, [0, 3], initial_ys(i));
    plot(x_sol, y_sol, "b-", "LineWidth", 2);
end

title("Slope field and solution curves for dy/dx = sin(xy)");
xlabel("x"); ylabel("y");
grid on;
```

KEY POINTS

- Normalize the arrows by dividing by their length L . Otherwise, steep slopes produce long arrows that overlap neighboring grid points, making the field hard to read.
- The third argument to `quiver` (0.5) scales the arrow length. Smaller values produce shorter, more compact arrows. Experiment with 0.3 to 0.7.
- Use element-wise multiplication (`.*`) in the ODE definition. The ODE function is called with vectorized inputs (X and Y are matrices), so `x * y` would fail — you need `x .* y`.
- Overlay solution curves by calling `ode45` for each initial condition. This shows how the slope field predicts the actual solution flow.

Verification — Back-Substitution

Never trust an unverified solution. The "Control of Error" discipline from the slides applies in Octave too: substitute your analytical solution back into the original ODE and confirm the equation is satisfied.

BACK-SUBSTITUTION WORKFLOW

Given a proposed solution $y(x)$, compute dy/dx symbolically, substitute into the ODE, and check that the left-hand side equals the right-hand side.

```
pkg load symbolic
syms x

% Proposed solution (from dsolve or by hand)
y_sol = (2/3)*x^4 + 4/(3*x^2);

% Compute dy/dx
dy_dx = diff(y_sol, x);

% Substitute into the LHS of the ODE: x*dy/dx + 2*y
lhs = x * dy_dx + 2 * y_sol;
lhs = simplify(lhs);

% The RHS of the ODE: 4*x^3
rhs = 4 * x^3;

% Check
if simplify(lhs - rhs) == 0
    disp("✓ VERIFIED: LHS = RHS");
else
    disp("✗ FAILED: LHS ≠ RHS");
end
```

NUMERICAL VS ANALYTICAL COMPARISON

A second verification approach: solve the ODE numerically with `ode45` and compare the results to your analytical solution at the same x -values. They should agree to within the numerical tolerance.

```
% Define the ODE in dy/dx = ... form
f = @(x, y) 4*x^2 - 2*y/x;

% Solve numerically
[x_num, y_num] = ode45(f, [1, 3], 2);

% Compare to analytical
fprintf("%-6s %12s %12s %12s\n", "x", "Numerical", "Analytical", "Error");
for i = 1:length(x_num)
    x_val = x_num(i);
    y_analytical = double(subs(y_sol, x, x_val));
    err = abs(y_num(i) - y_analytical);
    fprintf("%-6.2f %12.6f %12.6f %12.2e\n", ...
        x_val, y_num(i), y_analytical, err);
end
```

INTERPRETING THE ERROR

If the analytical solution is correct, the error column should show values on the order of $1e-6$ or smaller (limited by ode45's default tolerance). If you see errors of $1e-2$ or larger, either your analytical solution is wrong, or you have defined the ODE function f differently from the symbolic form. Re-derive and re-check.

COMMON PITFALLS

- Forgetting to declare y as a function of x . If you write `syms x y` (instead of `syms y(x) x`), then `diff(y, x)` returns 0 because Octave treats y as an independent variable. Always use the $y(x)$ form for ODE solving.
- Using `=` instead of `==` in ODE definitions. `ode = diff(y,x) == 2*x*y` creates a symbolic equation. `ode = diff(y,x) = 2*x*y` is a syntax error.
- Not loading the symbolic package. `pkg load symbolic` must appear at the top of every script that uses `dsolve`, `syms`, `int`, `diff` (for symbolic arguments), or `simplify`. Without it, these functions are undefined.
- Mixing numeric and symbolic arrays. `plot` requires numeric vectors. To plot a symbolic expression, use `fplot`, or convert with `double(subs(...))` first.